DEUKSIL — Commencer ici

Ce dossier contient **tout** ce qu'il faut pour configurer et mettre en service DEUKSIL sur Supabase.

> **Mise en ligne (Supabase + Gemini, site `https://deuksil.vercel.app`) :** `README_SUPABASE_GEMINI.md`.
>
> **Pour configurer, un seul fichier suffit :** `README_CONFIGURATION.md` — toutes les commandes et tous les réglages, de bout en bout, dans l'ordre. Les documents ci-dessous sont des compléments.

Documents complémentaires :

Ordre	Document	Pour quoi faire	Durée
1	`00_COMMENCER_ICI.md` (ce fichier)	savoir quoi ouvrir	2 min
2	`EXPLICATION_DES_DOSSIERS.md`	comprendre chaque dossier : ce qu'il contient, quand et comment l'utiliser (commandes complètes, fichier `.env`)	20 min
2 bis	`GUIDE_EQUIPE_BACKEND.md`	**les étapes 0 à 14, dans l'ordre**, avec commandes et contrôles	lecture 30 min
3	`FICHE_PARAMETRES.md`	noter chaque réglage (où le trouver, où le mettre, secret ou non) — à remplir pendant le guide	au fil de l'eau
4	`tests/RECETTE_MANUELLE_FORMULAIRES.md`	fiche à cocher (36 points) à l'étape 9	15 min
5	`README_SUPABASE.md`	référence détaillée (le guide y renvoie : « [§3.3] » = section 3.3)	au besoin
6	`docs/AUDIT_SECURITE.md` · `docs/PERFORMANCES.md` · `docs/INDEX.md`	rapports de sécurité, de performance, justification des index	au besoin

Le fichier `DEUKSIL_Dossier_Configuration.pdf` regroupe ces documents (commencer ici, explication des dossiers, guide, fiche des paramètres, recette) et la checklist finale en un seul PDF à imprimer.

Le parcours en une ligne

Projet de test → réglages → migrations → tâche quotidienne → fonction serveur → site → tests + recette → sauvegarde/restauration d'essai → **projet de production** → checklist → mise en ligne.

Où se trouve quoi

```
00_COMMENCER_ICI.md              ← vous êtes ici
EXPLICATION_DES_DOSSIERS.md      à quoi sert chaque dossier, comment l'utiliser
GUIDE_EQUIPE_BACKEND.md          étapes 0 à 14
FICHE_PARAMETRES.md              tous les réglages à faire
DEUKSIL_Dossier_Configuration.pdf  les documents ci-dessus en un PDF
README_SUPABASE.md               référence complète
.env.example                     variables à copier en .env (jamais commité)

supabase/
  migrations/      8 scripts SQL de la base           → étape 5
```

```
  functions/       fonctions serveur gerer-acces et assistant-ia → étape 7
  templates/       modèles des e-mails                → étape 4
  config.toml      réglages Supabase                  → étapes 3-4
  outils/          import du jeu de TEST, schéma      → étape 9
  sauvegarde/      sauvegarder.sh, restaurer.sh       → étapes 10 et 14
  outils/          …, vider-projet.sql / vider-fichiers.js / verifier-projet-vide.sql (remise à
zéro)
  (tests-local/    dans DEUKSIL_OUTILS_DE_TEST.zip : tests de sécurité, jeu de test, mesures →
étapes 9 et 11)

tests/
  RECETTE_MANUELLE_FORMULAIRES.md  fiche à cocher     → étape 9
  (*.js            dans DEUKSIL_OUTILS_DE_TEST.zip : tests navigateur → étape 9)

js/00-config.js    le seul fichier du site à renseigner → étape 8
index.html, css/, js/, img/   le site lui-même
docs/              rapports (sécurité, performances, index)
```

Trois règles

1. **D'abord un projet de test**, jamais de manipulation directe en production.
2. **Aucun secret** (clé `service_role`, mot de passe de base, secrets Google et SMTP) dans le code, dans Git, dans un message ou dans la fiche des paramètres.
3. **Un contrôle par étape** : on ne passe pas à l'étape suivante tant que le contrôle indiqué n'est pas bon.

Ce qui n'est pas encore fait

- Recherche des annonces côté serveur (à prévoir avant une forte croissance).
- Validation sur Supabase hébergé : Storage, fonction serveur, `config.toml` et restauration n'ont été testés que sur une réplique locale — les étapes 7, 9 et 10 du guide les valident.

Comprendre et utiliser les dossiers du kit DEUKSIL

Ce document explique, **dossier par dossier**, ce qu'il contient, **à quoi il sert**, **quand** l'utiliser et **comment**, avec des commandes complètes. Il complète le guide (`GUIDE_EQUIPE_BACKEND.md`), qui donne l'ordre des étapes.

Le principe en 30 secondes

DEUKSIL est fait de **deux parties** :

1. **Le site** (ce que voit l'utilisateur) : `index.html` , `css/` , `js/` , `img/` . Des fichiers à publier chez un hébergeur web. Rien à installer.
2. **Le serveur Supabase** (comptes, données, fichiers, sécurité) : tout ce qui est dans `supabase/` . Il est installé **dans votre projet Supabase** grâce aux scripts fournis.

Les scripts sont **déjà écrits et testés**. L'équipe ne les modifie pas : elle les **lance** sur son projet Supabase, avec **ses** valeurs (adresse du projet, clés), rangées une fois pour toutes dans un fichier `.env`.

Avant tout : le fichier `.env` (une seule fois)

Toutes les commandes ont besoin des valeurs de votre projet. Plutôt que de les recopier à chaque commande, on les met dans un fichier `.env`.

1. Créer le fichier à partir du modèle :

```
cp .env.example .env
```

2. Le remplir avec un éditeur de texte (valeurs du projet **de test** : Dashboard Supabase → Project Settings → API et → Database) :

```
SUPABASE_URL=https://abcdefghijklmnop.supabase.co
SUPABASE_ANON_KEY=eyJhbGciOi...            # clé « anon » (publique)
SUPABASE_SERVICE_ROLE_KEY=eyJhbGciOi...    # clé « service_role » (SECRÈTE)
DATABASE_URL=postgresql://postgres:MOT-DE-PASSE@db.abcdefghijklmnop.supabase.co:5432/postgres
DEUKSIL_ORIGINES=https://test.votre-domaine.sn
APP_URL=https://test.votre-domaine.sn/
MOT_DE_PASSE_TEST=Choisi-Par-Equipe-2026
DEUKSIL_TEST_AUTORISE=1
```

3. Le charger dans le terminal, au début de chaque séance de travail :

```
set -a; source .env; set +a
```

Ensuite, toutes les commandes de ce document fonctionnent **telles quelles**, sans rien recopier.

> `.env` contient des secrets : il est déjà exclu de Git (`.gitignore`). Ne jamais l'envoyer par message ni le joindre à un ticket. Un fichier `.env` par projet (test, production) : on peut les nommer `.env.test` et `.env.prod` et charger le bon (`source .env.test`).

Carte des dossiers

Dossier / fichier	Contenu	Utilisé quand	Qui
`supabase/migrations/`	8 scripts SQL : toute la base	une fois par projet, puis à chaque évolution	backend
`supabase/functions/gerer-acces/`	fonction serveur de création de comptes	une fois par projet, puis si elle change	backend
`supabase/templates/`	2 modèles d'e-mails	une fois par projet (copier-coller)	backend
`supabase/config.toml`	réglages de référence	base locale ; référence pour le tableau de bord	backend

Dossier / fichier	Contenu	Utilisé quand	Qui
`supabase/outils/`	2 outils : schéma pour le site ; import du jeu de test	à chaque changement de schéma ; projet de test	backend
`supabase/tests-local/`	tests de sécurité, jeu de test, mesures, réplique	projet de test	backend / QA
`supabase/sauvegarde/`	sauvegarde et restauration	chaque jour/semaine ; essai trimestriel	backend / exploitation
`tests/`	tests navigateur + fiche de recette	projet de test	QA
`js/00-config.js`	réglages publics du site	une fois par projet (test, prod)	frontend / backend
`docs/`	rapports (sécurité, performances, index)	lecture	tous

---

1. `supabase/migrations/` — la base de données

Ce que c'est. Huit fichiers SQL numérotés. Ensemble, ils créent **toute** la base : les 32 tables, les règles d'accès (qui voit quoi), les règles métier (numéros de factures, rappels, etc.), les index de performance.

Fichier	Contenu en clair
`…0100_base.sql`	extensions et outils de base
`…0200_tables.sql`	les 32 tables (comptes, biens, baux, loyers, factures…)
`…0300_comptes_et_droits.sql`	création automatique du profil à l'inscription, protection des comptes
`…0400_rls.sql`	les règles d'accès : chacun ne voit que ses données
`…0500_fonctions_api.sql`	fonctions appelées par le site (annuaire, notifications…)
`…0600_stockage.sql`	dossiers de fichiers (photos, avatars, documents privés)
`…0700_regles_et_rappels.sql`	numérotation, rappels quotidiens, règles de visites, de messages…
`…0800_performances.sql`	optimisations et index

Comment l'utiliser (étape 5 du guide) — on ne lance **pas** les fichiers un par un, la commande Supabase s'en charge, dans l'ordre :

```
supabase link --project-ref abcdefghijklmnop     # une fois : relier le dossier au projet
supabase db push                                 # applique les 8 fichiers dans l'ordre
```

Vérifier dans l'éditeur SQL du tableau de bord : `select count(*) from pg_tables where schemaname = 'public';` doit répondre **32**.

Règle importante : un fichier déjà appliqué ne se modifie **jamais**. Pour faire évoluer la base, on crée un nouveau fichier (`supabase migration new nom_du_changement`), puis `supabase db push` .

---

2. `supabase/functions/gerer-acces/` — la fonction serveur

Ce que c'est. Un petit programme qui tourne chez Supabase. Il crée les comptes qu'un propriétaire ou une agence ouvre pour quelqu'un d'autre (membre d'équipe, locataire, propriétaire mandant). Il est le

seul à utiliser la clé secrète `service_role`, **côté serveur**, jamais dans le navigateur.

Comment l'utiliser (étape 7) :

```
supabase functions deploy gerer-acces
supabase secrets set DEUKSIL_ORIGINES=$DEUKSIL_ORIGINES
```

Vérifier : `curl -i -X POST $SUPABASE_URL/functions/v1/gerer-acces` doit répondre **401** (« connexion requise ») : c'est normal, la fonction refuse les inconnus.

---

3. `supabase/templates/` — les e-mails

Ce que c'est. Deux pages HTML : l'e-mail de confirmation d'inscription et celui de mot de passe oublié. Chacun affiche un **code à 6 chiffres**.

Comment l'utiliser (étape 4) : pas de commande. Dans le tableau de bord → Authentication → Email Templates : * « Confirm signup » : coller le contenu de `confirmation.html` ; objet : `Confirmez votre adresse DEUKSIL` ; * « Reset password » : coller `recovery.html` ; objet : `Votre code DEUKSIL`.

---

4. `supabase/config.toml` — les réglages de référence

Ce que c'est. La liste des réglages Supabase recommandés (sécurité des mots de passe, e-mails, API…).

Comment l'utiliser : sur un projet hébergé, ces réglages se font **à la main** dans le tableau de bord, en suivant la `FICHE_PARAMETRES.md` (rubriques B et C). Le fichier sert aussi si l'équipe veut une base Supabase locale (`supabase start`, nécessite Docker).

---

5. `supabase/outils/` — deux outils

Outil	À quoi il sert	Quand
`importer-json.js`	charge le **jeu de données de test** (29 comptes fictifs, biens, baux…)	projet de **test** uniquement, avant les tests
`generer-schema-front.js`	met à jour `js/02c-schema.js`, la description des tables utilisée par le site	après chaque nouvelle migration qui touche les tables

Commandes (après `source .env`, projet de test) :

```
npm install @supabase/supabase-js@2.45.4 pg            # une fois
MOT_DE_PASSE=$MOT_DE_PASSE_TEST node supabase/outils/importer-json.js \
  supabase/tests-local/jeu-de-test.json supabase/tests-local/comptes-test.json

node supabase/outils/generer-schema-front.js           # après un changement de tables
```

> **Jamais `importer-json.js` sur la production** : il crée des comptes de test.

---

6. `supabase/tests-local/` — tests et données de test

> 📦 **Les outils de test ne sont pas dans ce dossier** : le projet est livré **vide** (aucun compte, aucun bien, aucune donnée). Les tests automatiques et le jeu de données de test (`supabase/tests-local/` , `tests/*.js`) sont dans **`DEUKSIL_OUTILS_DE_TEST.zip`** , à décompresser à la racine du projet **uniquement sur le poste qui teste le projet Supabase de TEST**. Ne jamais publier ces fichiers sur l'hébergement.

Élément	Rôle
`securite-rls.test.js`	**65 tests de sécurité** : se connecte avec 11 vrais comptes et vérifie que chacun ne voit et ne modifie que ce qu'il doit
`jeu-de-test.json`	données fictives (adresses `@deuksil.test`)
`comptes-test.json`	liste des comptes de test (rôle, e-mail), produite par l'import
`perf/`	mesures de performance (facultatif, jamais en production)
`replique/`	copie locale de Supabase utilisée pour le développement de ce kit (facultatif)
`resultats/`	résultats des derniers tests

Commande (étape 9) :

```
node supabase/tests-local/securite-rls.test.js
```

Attendu : `65/65 tests réussis` .

---

7. `tests/` — tests du site et recette

Élément	Rôle	Commande
`acces-comptes.js`	ouverture d'accès locataire / équipe / mandant	`node tests/acces-comptes.js` → 14/14
`sonde-pages.js`	ouvre toutes les pages des 9 profils, cherche les erreurs	`node tests/sonde-pages.js` → « OK » partout
`ecritures-auto.js`	vérifie que le site n'écrit rien sans action de l'utilisateur	`node tests/ecritures-auto.js` → aucune ligne
`xss-sonde.js`	tente d'injecter du code dans les données (**modifie la base : en dernier**)	`node tests/xss-sonde.js` → « Aucune exécution »
`RECETTE_MANUELLE_FORMULAIRES.md`	fiche à cocher : inscription, code e-mail, mot de passe oublié, photos, contact	à la main, 15 min

Prérequis une fois : `npm install playwright pg && npx playwright install chromium` .

---

8. `supabase/sauvegarde/` — sauvegarder et restaurer

Ce que c'est. Deux scripts : * `sauvegarder.sh` copie **toute la base** (comptes compris) **et tous les fichiers** (photos, documents) dans un dossier daté ; * `restaurer.sh` remet une sauvegarde dans un

projet **neuf** (il refuse un projet qui contient déjà des comptes, pour protéger la production).

Sauvegarder (après `source .env`) :

```
cd supabase/sauvegarde
npm install                    # une fois
./sauvegarder.sh               # crée sauvegardes/AAAAMMJJ-HHMM/
```

Le dossier obtenu contient des **données personnelles** : le chiffrer (`gpg -c`) et le ranger hors du projet.

Restaurer (dans un projet neuf où les migrations ont été appliquées) :

```
DATABASE_URL_CIBLE="postgresql://postgres:…@db.<ref-neuf>.supabase.co:5432/postgres" \
SUPABASE_URL="https://<ref-neuf>.supabase.co" \
SUPABASE_SERVICE_ROLE_KEY="<clé service_role du projet neuf>" \
CONFIRMER=oui ./restaurer.sh sauvegardes/AAAAMMJJ-HHMM
```

Attendu à la fin : `VÉRIFICATION OK : mêmes comptages qu'à la sauvegarde.`

---

9. `js/00-config.js` — brancher le site sur Supabase

Ce que c'est. Le **seul** fichier du site à modifier. Il dit au site à quel projet Supabase parler.

Comment : l'ouvrir dans un éditeur et remplir :

```
supabaseUrl: 'https://abcdefghijklmnop.supabase.co',
supabaseAnonKey: 'eyJhbGciOi...',     // clé ANON uniquement — jamais service_role
googleActif: false,
siteUrl: 'https://test.votre-domaine.sn',
editeur: { raisonSociale: '…', ninea: '…', /* mentions légales */ }
```

Puis publier les fichiers du site chez l'hébergeur.

---

10. `docs/` — les rapports

Document	À lire si…
`AUDIT_SECURITE.md`	on veut savoir ce qui a été sécurisé, testé, et ce qui reste à surveiller
`PERFORMANCES.md`	on veut les mesures avant / après et les limites connues
`INDEX.md`	on se demande pourquoi un index existe

Résumé : une séance type sur le projet de test

```
cp .env.example .env.test          # une fois, puis remplir
set -a; source .env.test; set +a   # à chaque séance

supabase link --project-ref <ref>  # une fois
supabase db push                   # base
supabase functions deploy gerer-acces
supabase secrets set DEUKSIL_ORIGINES=$DEUKSIL_ORIGINES
```

```
# → régler e-mails et adresse du site dans le tableau de bord (FICHE_PARAMETRES)
# → remplir js/00-config.js et publier le site de test

MOT_DE_PASSE=$MOT_DE_PASSE_TEST node supabase/outils/importer-json.js \
  supabase/tests-local/jeu-de-test.json supabase/tests-local/comptes-test.json
node supabase/tests-local/securite-rls.test.js
node tests/acces-comptes.js
node tests/sonde-pages.js
node tests/ecritures-auto.js
# → fiche RECETTE_MANUELLE_FORMULAIRES.md à la main
node tests/xss-sonde.js             # en dernier

cd supabase/sauvegarde && npm install && ./sauvegarder.sh
```

Questions fréquentes

Faut-il modifier des scripts ? Non. Seuls `.env` (vos valeurs) et `js/00-config.js` (site) se remplissent.

Que faire si une commande échoue ? Lire le message, consulter `README_SUPABASE.md` §18 (erreurs fréquentes), ne pas passer à l'étape suivante.

Peut-on lancer les tests sur la production ? Non. Les tests et l'import créent ou modifient des données : projet de test uniquement.

Où sont les mots de passe et clés ? Dans le tableau de bord Supabase et dans `.env` (local, jamais partagé). Nulle part ailleurs.

DEUKSIL — Guide de mise en service pour l'équipe backend

À suivre **dans l'ordre**. Chaque étape indique : le but, les fichiers, les commandes, et le contrôle qui dit « c'est bon, on passe à la suivante ». Le détail de chaque réglage est dans `README_SUPABASE.md` (numéros de section entre crochets, ex. [§3.3]).

> **Règle n° 1** : tout se fait d'abord sur un projet Supabase **de test**. La production ne vient qu'à l'étape 12. **Règle n° 2** : la clé `service_role`, le mot de passe de la base, les secrets Google et SMTP ne vont **jamais** dans le code, dans Git ou dans un message. Ils se saisissent dans le tableau de bord Supabase ou dans un `.env` local (ignoré par Git).

Étape 0 — Ce que contient l'archive

Élément	À quoi il sert	Utilisé à l'étape
`GUIDE_EQUIPE_BACKEND.md`	ce guide	toutes
`README_SUPABASE.md`	référence complète (19 sections)	toutes
`supabase/migrations/` (8 fichiers `.sql`)	structure de la base, sécurité (RLS), fonctions, index	5

Élément	À quoi il sert	Utilisé à l'étape
`supabase/functions/gerer-acces/`	fonction serveur : création des comptes ouverts par un tiers	7
`supabase/config.toml`	réglages Supabase (local + référence)	3, 4
`supabase/templates/`	modèles des e-mails (code à 6 chiffres)	4
`.env.example`	liste des variables, sans valeur	2
`js/00-config.js`	seul fichier à renseigner côté site	8
`supabase/outils/`	génération du schéma pour l'interface ; import du jeu de **test**	9
`supabase/tests-local/`	tests de sécurité, jeu de test, mesures de performance, réplique locale	9, 11
`tests/`	tests navigateur	9
`supabase/sauvegarde/`	sauvegarde et restauration (base + fichiers)	10, 14
`docs/AUDIT_SECURITE.md` , `docs/PERFORMANCES.md` , `docs/INDEX.md`	rapports	1

Étape 1 — Lire (≈ 1 h)

1. Ce guide en entier.
2. `README_SUPABASE.md` §1 (architecture), §8 (RLS), §10 (secrets), §19 (checklist).
3. `docs/AUDIT_SECURITE.md` §8 (risques restants) : décisions à prendre avec le responsable produit.

Contrôle : l'équipe sait où va chaque secret et sait que la base décide des droits (jamais le navigateur).

Étape 2 — Installer les outils

Outil	Commande de vérification
Supabase CLI	`supabase --version`
PostgreSQL client (psql, pg_dump) **≥ version du serveur Supabase**	`psql --version`
Node.js 18+	`node --version`
Playwright + Chromium (tests)	`npx playwright --version`
(facultatif) Docker, pour `supabase start`	`docker --version`

Copier `.env.example` en `.env` (jamais commité) et le remplir au fur et à mesure.

Contrôle : toutes les commandes répondent.

Étape 3 — Créer le projet Supabase de TEST

1. Dashboard Supabase → *New project* : nom `deuksil-test` , région proche des utilisateurs, mot de passe de base fort (gestionnaire de secrets).

2. Noter dans `.env` : `SUPABASE_URL`, `SUPABASE_ANON_KEY`, `SUPABASE_SERVICE_ROLE_KEY`, `DATABASE_URL` (Project Settings → API / Database).
3. Lier le dépôt : `supabase link --project-ref <ref-test>`.

Contrôle : `supabase projects list` montre le projet lié.

Étape 4 — Régler l'API et l'authentification [§3.2, §3.3]

Dans le tableau de bord du projet de test :

- **API** : schémas exposés = `public` **uniquement** ; *Max rows* = 1000.
- **Authentication** : Site URL et Redirect URLs (adresse où le site de test sera servi) ; confirmation d'e-mail **activée** ; mots de passe 8 caractères, lettres + chiffres ; OTP 900 s ; inscriptions anonymes désactivées.
- **E-mails** : SMTP personnalisé (le SMTP intégré est limité) ; coller `supabase/templates/confirmation.html` et `recovery.html` dans les modèles « Confirm signup » et « Reset password ».
- **Google** : laisser désactivé pour le test (ou suivre [§3.3]).

Contrôle : une inscription manuelle de test reçoit bien un e-mail avec un code à 6 chiffres.

Étape 5 — Appliquer les migrations [§12]

```
supabase db push            # applique les 8 migrations dans l'ordre
```

Contrôles dans l'éditeur SQL :

```
select count(*) from pg_tables where schemaname = 'public';                      -- 32
select count(*) from pg_tables where schemaname = 'public' and not rowsecurity;   -- 0
select count(*) from pg_policies where schemaname = 'public';                    -- 166
select id, public from storage.buckets order by id;   -- avatars t, documents f, photos t
```

Si une migration échoue : lire le message, corriger **dans une nouvelle migration** si la précédente est déjà passée, ne jamais modifier une migration appliquée.

Contrôle : les quatre résultats ci-dessus.

Étape 6 — Activer les tâches quotidiennes (pg_cron) [§3.4]

1. Dashboard → Integrations → Cron (ou Database → Extensions → `pg_cron`).
2. Si la migration 0700 n'a pas pu planifier la tâche (message `NOTICE` pendant le `db push`) : `sql select cron.schedule('deuksil-rappels', '0 7 * * *', 'select app.generer_rappels()');`
3. Essai immédiat : `select * from app.generer_rappels();` (deux fois : la seconde ne crée rien).

Contrôle : `select jobname, schedule, active from cron.job;` montre `deuksil-rappels` actif.

Étape 7 — Déployer la fonction serveur [§3.6]

```
supabase functions deploy gerer-acces
supabase secrets set DEUKSIL_ORIGINES=<adresse du site de test>
```

Contrôle : `curl -i -X POST https://<ref>.supabase.co/functions/v1/gerer-acces` répond **401** (connexion requise) — c'est le comportement attendu.

Étape 8 — Brancher le site [§3.7, §11]

1. `js/00-config.js` : `supabaseUrl`, `supabaseAnonKey` (clé **anon**), `siteUrl`, `googleActif: false`.
2. Si le projet a un domaine personnalisé : l'ajouter à `connect-src` dans la balise CSP d'`index.html`.
3. Servir les fichiers en HTTPS (hébergement statique) à l'adresse déclarée à l'étape 4.

Contrôle : la page d'accueil s'ouvre sans « Configuration incomplète » ; inscription → code reçu → connexion → déconnexion fonctionnent.

Étape 9 — Tests sur le projet de TEST [§14]

1. Jeu de test (projet de **test** uniquement) : `bash DATABASE_URL=… SUPABASE_URL=… SUPABASE_SERVICE_ROLE_KEY=… MOT_DE_PASSE=<mot de passe de test> \ node supabase/outils/importer-json.js supabase/tests-local/jeu-de-test.json supabase/tests-local/comptes-test.json`
2. Dans cet ordre (variables : `SUPABASE_URL`, `SUPABASE_ANON_KEY`, `MOT_DE_PASSE_TEST`, `APP_URL` = adresse du site de test, `DEUKSIL_TEST_AUTORISE=1` car le projet n'est pas local) :

Ordre	Commande	Attendu
1	`node supabase/tests-local/securite-rls.test.js`	65/65
2	`node tests/acces-comptes.js`	14/14
3	`node tests/sonde-pages.js`	9 profils « OK »
4	`node tests/ecritures-auto.js`	aucune ligne (pas d'écriture automatique)
5	**Recette manuelle des formulaires** : imprimer et suivre `tests/RECETTE_MANUELLE_FORMULAIRES.md` (≈ 15 min, 36 points à cocher, deux adresses e-mail de test)	36/36 OK, fiche signée
6	`node tests/xss-sonde.js` (`DATABASE_URL` du test) — **modifie les données** : à lancer en dernier, ou sur un second projet de test	« Aucune exécution »

Pourquoi à la main : les codes d'inscription et de mot de passe oublié arrivent par e-mail. Le test automatique `tests/formulaires.js` sait les lire uniquement sur la réplique locale ; sur un projet hébergé, une personne reçoit les e-mails et suit la fiche. On vérifie ainsi, en plus, que les e-mails partent vraiment.

3. Après `xss-sonde.js`, repartir d'une base propre (nouveau projet de test, ou `supabase db reset --linked` **sur le projet de test seulement**).

Contrôle : tous les résultats attendus. Un écart = s'arrêter et analyser (README_SUPABASE.md §18).

Étape 10 — Sauvegarde et restauration (une fois, avant toute vraie donnée) [§17]

```
cd supabase/sauvegarde && npm install
# a) sauvegarder le projet de test
DATABASE_URL=… SUPABASE_URL=… SUPABASE_SERVICE_ROLE_KEY=… ./sauvegarder.sh
# b) créer un 2e projet vide « deuksil-restauration », y appliquer les migrations (étape 5)
# c) restaurer dedans
DATABASE_URL_CIBLE=… SUPABASE_URL=<restauration> SUPABASE_SERVICE_ROLE_KEY=<clé restauration>
CONFIRMER=oui \
  ./restaurer.sh sauvegardes/<date>
```

Contrôle : « VÉRIFICATION OK : mêmes comptages », connexion d'un compte de test sur le projet restauré, document privé lisible par son propriétaire. Noter le temps total : c'est votre délai de remise en service. Supprimer ensuite le projet de restauration.

Étape 11 — (facultatif) Mesures de performance

Sur la réplique locale ou un projet jetable, jamais en production : `supabase/tests-local/perf/` (README §15, docs/PERFORMANCES.md).

Étape 12 — Projet de PRODUCTION

Refaire les étapes **3 à 8** sur un nouveau projet `deuksil-prod`, **sans** l'étape 9.1 (aucun jeu de test), puis :

1. Premier administrateur : inscription normale, puis dans l'éditeur SQL : `update public.profiles set role = 'admin' where email = '<adresse>';`
2. MFA de l'administrateur (TOTP), puis : `update app.reglages set valeur = 'oui' where cle = 'admin_mfa_obligatoire';`
3. En-têtes HTTP de l'hébergement : `Strict-Transport-Security`, `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, `Referrer-Policy: strict-origin-when-cross-origin` [§16].
4. Mentions légales : bloc `editeur` de `js/00-config.js`.

Étape 13 — Mise en ligne

Cocher **toute** la checklist `README_SUPABASE.md` §19. Puis parcours manuel : inscription, publication d'un bien avec photos, demande de visite, message, document privé, déconnexion.

Étape 14 — Exploitation courante

Fréquence	Action
chaque jour	vérifier `cron.job_run_details` (tâche `deuksil-rappels`)
chaque semaine (ou chaque jour)	`supabase/sauvegarde/sauvegarder.sh`, dossier chiffré et copié hors du projet
chaque trimestre	refaire une restauration d'essai (étape 10)
chaque mois	index inutilisés et requêtes lentes (docs/INDEX.md, « Suivi en production »)

Fréquence	Action
à chaque évolution	nouvelle migration → projet de test → étape 9 → production ; `node supabase/outils/generer-schema-front.js` si le schéma change

Ce qui reste ouvert (à planifier)

1. **Recherche des annonces côté serveur** : aujourd'hui l'interface charge tout le catalogue public à l'ouverture ; à faire avant une forte croissance (docs/PERFORMANCES.md, « Ce qui reste »).
2. **Validations sur Supabase hébergé** : Storage, fonction Edge, `config.toml` et restauration (`session_replication_role`) n'ont été testés que sur la réplique locale — les étapes 7, 9 et 10 les valident.
3. Décisions produit listées dans `docs/AUDIT_SECURITE.md` §8 (annuaire, notifications, MFA, offre de sauvegarde).

Fiche des paramètres de configuration

Tous les réglages à faire, au même endroit : d'où vient chaque valeur, où elle va, et si elle est **secrète**.

- Colonne « Valeur » : remplir **uniquement** les valeurs **publiques** (cette fiche peut circuler dans l'équipe).
- Une valeur **SECRÈTE** ne s'écrit **jamais** ici, ni dans Git, ni dans un message : seulement dans le tableau de bord Supabase, dans `supabase secrets set` , ou dans le fichier `.env` local (ignoré par Git). Noter seulement *où* elle est rangée (ex. « coffre de l'équipe »).

Remplir une fiche pour le projet de **test** et une autre pour la **production**.

Projet : ☐ test ☐ production Rempli par : ____ Date : _ / _ / _____

A. Projet Supabase (étape 3)

#	Paramètre	Où le trouver	Où il va	Secret ?	Valeur / emplacement
A1	Nom du projet	à choisir (`deuksil-test` , `deuksil-prod`)	Dashboard	non	
A2	Référence du projet (`ref`)	Project Settings → General	`supabase link --project-ref`	non	
A3	Région	choisie à la création	`js/00-config.js` → `editeur.regionDonnees`	non	
A4	URL du projet `https://<ref>.supabase.co`	Project Settings → API	`.env` `SUPABASE_URL` , `js/00-config.js` `supabaseUrl`	non	
A5	Clé **anon / publishable**	Project Settings →	`.env` `SUPABASE_ANON_KEY` , `js/00-config.js`	non (publique par	

#	Paramètre	Où le trouver	Où il va	Secret ?	Valeur / emplacement
		API	`supabaseAnonKey`	conception)	
A6	Clé **service_role**	Project Settings → API	`.env` local uniquement (outils, tests, sauvegarde)	**SECRÈTE**	rangée :
A7	Mot de passe de la base	choisi à la création	`.env` `DATABASE_URL`	**SECRET**	rangé :
A8	Chaîne de connexion `DATABASE_URL`	Project Settings → Database	`.env`	**SECRÈTE** (contient le mot de passe)	rangée :

B. API (étape 4)

#	Paramètre	Valeur attendue	Où	Fait
B1	Schémas exposés	`public` uniquement	Settings → API / Data API	☐
B2	Max rows	`1000`	Settings → API	☐

C. Authentification (étape 4)

#	Paramètre	Valeur attendue / à remplir	Où	Secret ?	Fait
C1	Site URL	adresse du site : ____	Authentication → URL Configuration	non	☐
C2	Redirect URLs	`<adresse du site>/**` : ____	idem	non	☐
C3	Confirmation d'e-mail	activée	Authentication → Providers → Email	non	☐
C4	Longueur minimale du mot de passe	8, lettres + chiffres	Authentication → Policies / Providers	non	☐
C5	Durée du code (OTP)	900 s	Providers → Email	non	☐
C6	Inscriptions anonymes	désactivées	Authentication → Settings	non	☐
C7	Modèle « Confirm signup »	contenu de `supabase/templates/confirmation.html` ; objet « Confirmez votre adresse DEUKSIL »	Authentication → Email Templates	non	☐
C8	Modèle « Reset password »	contenu de `supabase/templates/recovery.html` ; objet « Votre code DEUKSIL »	idem	non	☐

D. Envoi des e-mails — SMTP (étape 4)

#	Paramètre	Où le trouver	Où il va	Secret ?	Valeur / emplacement
D1	Fournisseur SMTP	à choisir	—	non	

#	Paramètre	Où le trouver	Où il va	Secret ?	Valeur / emplacement
D2	Serveur (host) et port	fournisseur SMTP	Authentication → SMTP Settings	non	
D3	Utilisateur SMTP	fournisseur SMTP	idem	non	
D4	Mot de passe SMTP	fournisseur SMTP	idem	**SECRET**	rangé :
D5	Adresse d'expédition	ex. `noreply@votre-domaine.sn`	idem	non	
D6	Nom d'expéditeur	`DEUKSIL`	idem	non	☐

E. Connexion Google (facultative)

#	Paramètre	Où le trouver	Où il va	Secret ?	Valeur / emplacement
E1	Activer maintenant ?	décision	—	—	☐ oui ☐ plus tard
E2	URI de redirection à déclarer chez Google	`https://<ref>.supabase.co/auth/v1/callback`	Google Cloud → Identifiants	non	
E3	ID client OAuth	Google Cloud	Authentication → Providers → Google	non	
E4	Secret client OAuth	Google Cloud	idem	**SECRET**	rangé :
E5	`googleActif`	`true` si activé, sinon `false`	`js/00-config.js`	non	

F. Base de données (étapes 5 et 6)

#	Paramètre	Valeur attendue	Contrôle	Fait
F1	Migrations appliquées	8 (`supabase db push`)	32 tables, 0 sans RLS, 166 politiques	☐
F2	Extension pg_cron	activée	Integrations → Cron	☐
F3	Tâche `deuksil-rappels`	`0 7 * * *` (07:00 UTC = Dakar)	`select * from cron.job;`	☐

G. Fonction serveur « gerer-acces » (étape 7)

#	Paramètre	Valeur	Où	Secret ?	Fait
G1	Déploiement	`supabase functions deploy gerer-acces`	CLI	—	☐
G2	`DEUKSIL_ORIGINES`	adresse(s) du site, séparées par des virgules : ____	`supabase secrets set`	non	☐

#	Paramètre	Valeur	Où	Secret ?	Fait
G3	`SUPABASE_URL`, `SUPABASE_ANON_KEY`, `SUPABASE_SERVICE_ROLE_KEY`	**ne rien faire** : fournis automatiquement par Supabase	—	—	—
G4	Assistant	`supabase functions deploy assistant-ia --no-verify-jwt`	CLI	—	☐
G5	`GEMINI_API_KEY`	clé Gemini (Google AI Studio) — **ne pas l'écrire ici**	`supabase secrets set`	**OUI**	☐
G6	`GEMINI_MODEL`	facultatif, défaut `gemini-flash-latest`	`supabase secrets set`	non	☐
G7	`GEMINI_PAYANT`	`oui` **seulement** si la facturation Gemini est activée (questions sur les factures, contrats, loyers…)	`supabase secrets set`	non	☐

H. Site — `js/00-config.js` (étape 8)

#	Champ	Valeur	Fait
H1	`supabaseUrl`	= A4	☐
H2	`supabaseAnonKey`	= A5 (**jamais** la clé service_role)	☐
H3	`siteUrl`	= C1	☐
H4	`googleActif`	= E5	☐
H5	`editeur.raisonSociale`		☐
H6	`editeur.formeJuridique`		☐
H7	`editeur.adresse`		☐
H8	`editeur.ninea`		☐
H9	`editeur.rccm`		☐
H10	`editeur.directeurPublication`		☐
H11	`editeur.email`		☐
H12	`editeur.hebergeurSite`		☐
H13	`editeur.regionDonnees`	= A3	☐
H14	CSP `connect-src` (`index.html`)	ajouter le domaine Supabase personnalisé s'il y en a un	☐

I. Hébergement du site (étape 12)

#	Paramètre	Valeur	Fait
I1	Hébergeur / adresse	____	☐
I2	HTTPS	obligatoire	☐
I3	En-têtes	`Strict-Transport-Security`, `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, `Referrer-Policy: strict-origin-when-cross-origin`	☐

J. Administration (étape 12, production)

#	Paramètre	Valeur	Fait
J1	E-mail du premier administrateur	____	☐
J2	Rôle `admin` attribué (éditeur SQL)	`update public.profiles set role = 'admin' where email = '…';`	☐
J3	Double authentification (MFA) de l'administrateur	activée	☐
J4	MFA obligatoire pour les admins	`update app.reglages set valeur = 'oui' where cle = 'admin_mfa_obligatoire';`	☐

K. Sauvegardes (étapes 10 et 14)

#	Paramètre	Valeur / décision	Fait
K1	Offre Supabase et sauvegardes incluses	vérifier Database → Backups : ____	☐
K2	Fréquence de nos exports (`sauvegarder.sh`)	☐ chaque jour ☐ chaque semaine	☐
K3	Lieu de conservation (chiffré, hors du projet)	____	☐
K4	Clé de chiffrement des sauvegardes	**SECRÈTE** — rangée :	☐
K5	Restauration d'essai réussie le	**_ / _ / __ — durée : ____**	☐

L. Tests (projet de test uniquement, étape 9)

#	Paramètre	Valeur	Secret ?
L1	`APP_URL` (site de test)		non
L2	`MOT_DE_PASSE_TEST` (comptes du jeu de test)	choisi par l'équipe	à garder dans l'équipe, jamais en production
L3	`DEUKSIL_TEST_AUTORISE`	`1` (projet de test hébergé)	non
L4	Adresses e-mail de recette (A client, B agence)		non

Fiche de recette manuelle — formulaires (projet Supabase de TEST)

À faire à l'**étape 9** du guide, sur le **projet de test**, jamais sur la production. Durée : environ 15 minutes. Elle remplace `tests/formulaires.js`, qui ne peut pas lire les e-mails d'un projet hébergé.

Préparer - deux adresses e-mail de test **auxquelles vous avez accès** (A pour le client, B pour l'agence). Astuce Gmail : `vous+client@gmail.com`, `vous+agence@gmail.com` arrivent dans la même boîte ; - un téléphone ou un ordinateur, navigateur en navigation privée ; - l'adresse du site de test.

Testeur : ____ Date : **_ / _ / __ Navigateur : _____**

Cocher **OK** ou **KO**. En cas de KO, noter le message affiché et faire une capture d'écran.

---

1. Accueil (2 min)

#	Action	Résultat attendu	OK	KO
1.1	Ouvrir le site	page d'accueil complète, pas de « Configuration incomplète »	☐	☐
1.2	Cliquer « Publier un bien »	formulaire d'inscription, profil **Propriétaire** coché	☐	☐
1.3	Revenir, cliquer « Créer un compte agence »	profil **Agence** coché, champs de l'agence visibles	☐	☐
1.4	Lire le formulaire	aucune mention de « démonstration », « fictif » ou « paiement »	☐	☐

2. Inscription d'un client avec l'adresse A (4 min)

Ouvrir « Créer un compte », choisir **Client**, puis :

#	Action	Résultat attendu	OK	KO
2.1	Étape 2 : e-mail `pas-une-adresse`	message « Adresse e-mail invalide », on reste à l'étape 2	☐	☐
2.2	Téléphone `12345`	message « téléphone invalide »	☐	☐
2.3	CNI `123`	message « 13 chiffres »	☐	☐
2.4	Corriger : adresse **A**, téléphone `77 123 45 67`, CNI de 13 chiffres	passage à l'étape 3	☐	☐
2.5	Mot de passe `abcdefgh` (sans chiffre)	refusé (« trop faible » / chiffre manquant)	☐	☐
2.6	Mot de passe et confirmation différents	« ne correspondent pas »	☐	☐
2.7	Ne pas cocher les conditions d'utilisation	refusé (« conditions »)	☐	☐
2.8	Mot de passe valide (8 caractères, lettre et chiffre), conditions cochées, valider	page « Vérifiez votre adresse e-mail »	☐	☐
2.9	Ouvrir la boîte A	e-mail « Confirmez votre adresse DEUKSIL » avec un **code à 6 chiffres**, reçu en moins de 2 min (regarder aussi les indésirables)	☐	☐
2.10	Taper un faux code `000000`	refusé, pas de connexion	☐	☐
2.11	Taper le bon code	connecté, arrivée sur l'espace client	☐	☐

Délai de réception de l'e-mail : _____ secondes.

3. Connexion (2 min)

#	Action	Résultat attendu	OK	KO
3.1	Se déconnecter, ouvrir « Se connecter »	champ « Adresse e-mail », lien « Mot de passe oublié ? »	☐	☐
3.2	Adresse A + mauvais mot de passe	message « incorrect », pas de connexion	☐	☐
3.3	Adresse A + bon mot de passe	connecté	☐	☐

4. Mot de passe oublié (3 min)

#	Action	Résultat attendu	OK	KO
4.1	Se déconnecter → « Mot de passe oublié ? » → adresse A	« Code envoyé » (le message ne dit pas si le compte existe)	☐	☐
4.2	Boîte A	e-mail « Votre code DEUKSIL » avec un code à 6 chiffres	☐	☐
4.3	« J'ai déjà un code » : adresse A, code, nouveau mot de passe ×2	message « Mot de passe modifié », arrivée sur l'espace du compte	☐	☐
4.4	Se déconnecter, se reconnecter avec le **nouveau** mot de passe	connecté	☐	☐
4.5	Essayer l'**ancien** mot de passe	refusé	☐	☐

5. Inscription d'une agence avec l'adresse B (2 min)

#	Action	Résultat attendu	OK	KO
5.1	« Créer un compte agence » : nom de l'agence, adresse, NINEA, prénom, nom, adresse **B**, téléphone, CNI, mot de passe, conditions	page « Vérifiez votre adresse e-mail »	☐	☐
5.2	Taper le code reçu sur B	arrivée sur le **tableau de bord de l'agence**	☐	☐

6. Photos d'une annonce (2 min, connecté en agence)

#	Action	Résultat attendu	OK	KO
6.1	« Ajouter un bien »	la zone photos annonce « jusqu'à 20 photos »	☐	☐
6.2	Choisir **23** photos d'un coup	**20** gardées (compteur « 20/20 »), l'ajout de photos se désactive	☐	☐
6.3	Compléter et enregistrer le bien, puis le rouvrir	les photos s'affichent	☐	☐

7. Formulaire de contact (1 min, déconnecté)

#	Action	Résultat attendu	OK	KO
7.1	Pied de page → « Nous écrire »	page Contact	☐	☐
7.2	Message `court`	refusé (« trop court »)	☐	☐
7.3	Message complet, envoyer	« Message envoyé »	☐	☐
7.4	Se connecter en administrateur → notifications	une notification « Contact · … » avec le message	☐	☐

Contrôles dans la base (éditeur SQL du projet de test)

Remplacer les adresses par celles utilisées.

```
-- 2 : le client existe, avec sa CNI et la date d'acceptation des conditions
select role, cni, cgu_acceptees_le is not null as cgu from public.profiles where email = 'adresse-
```

```
A';
-- 5 : l'agence existe avec son nom et son NINEA
select role, agency_name, ninea from public.profiles where email = 'adresse-B';
-- 7 : le message de contact est enregistré, statut « nouveau »
select statut, subject, created_at from public.contact_messages order by created_at desc limit 1;
-- aucun mot de passe en clair nulle part dans public
select count(*) from information_schema.columns where table_schema = 'public' and column_name
ilike '%password%';  -- 0
```

Contrôle	OK	KO
profil client (role `client`, CNI, cgu = true)	☐	☐
profil agence (role `agency`, nom, NINEA)	☐	☐
message de contact (statut `nouveau`)	☐	☐
aucune colonne de mot de passe (0)	☐	☐

Résultat

Nombre de OK : **__ / 36 Nombre de KO : ____**

☐ **Recette validée** (tout est OK) ☐ **Recette refusée** (au moins un KO : à corriger puis refaire la fiche)

Signature : ____

En cas de KO : README_SUPABASE.md §18 (erreurs fréquentes). E-mails non reçus → vérifier le SMTP personnalisé (Authentication → SMTP) et les indésirables.